

Adaptive Neural Graph (ANG)

Een zelfstructurerend neuraal netwerk zonder lagen en zonder backpropagation

Frank Jacobs

Correspondentie: frljacobs@gmail.com

Preprint, versie 1.2 · 9 september 2026 · niet peer-reviewed

Bij dit onderzoek is een generatief AI-hulpmiddel gebruikt, en wel in aanzienlijke mate. Wat het precies heeft gedaan staat in de verklaring achterin; die verklaring hoort bij het werk en niet in de kleine lettertjes.

Samenvatting

Wij beschrijven de Adaptive Neural Graph (ANG): een neuraal rekenmodel waarin de topologie geen vaste lagen kent maar een gerichte graaf is die zichzelf tijdens het leren herstructureert. Elke knoop draagt een soort die bepaalt welke verbindingen zijn toegestaan — invoer, worker, reflex en geheugen — en die soort ligt niet vast: er kunnen neuronen bijgroeien, zij komen neutraal binnen, en zij krijgen een soort toegewezen op grond van de bedrading die zij feitelijk hebben ontwikkeld. Leren gebeurt zonder backpropagation. De uitvoerknopen vormen een Bernoulli-beleid waarvan de score-functie exact bekend is; voor de verborgen knopen wordt de gradiënt geschat met node-perturbatie, uit het verschil tussen een schone en een verstoorde doorrekening van dezelfde tijdstap. De afwijking van de beloning ten opzichte van een lopende basislijn moduleert een eligibility trace. Drie begrenzingsen — een maximale stap per keer, een damping die toeneemt naarmate een gewicht zijn plafond nadert, en een vervaging per poging — zorgen dat verbindingen geleidelijk ontstaan, versterken en verdwijnen in plaats van abrupt. Dit document beschrijft het model, de leerregel, de structurele plasticiteit en de meetinstrumenten volledig, met een referentiemeting over twaalf onafhankelijke breinzaden en een numerieke controle van de leerregel zelf. Die controle laat zien dat de exacte score-functie voor de uitvoerknopen de gradiënt nauwkeurig volgt, dat de node-perturbatieschatter de goede richting aanwijst maar veel ruwer, en dat de twee delen van de update niet op dezelfde schaal staan. De ablaties en de basislijnen volgen in een latere versie.

Trefwoorden: zelfstructurerende netwerken · reward-gemoduleerde plasticiteit · node-perturbatie · beleidsgradiënt · structurele plasticiteit · interpreteerbaarheid

1 Inleiding

1.1 Waarom geen lagen

Het gangbare neurale netwerk is een gestapelde functie: de invoer gaat door laag één, het resultaat door laag twee, enzovoort tot de uitvoer. Die vorm is geen eigenschap van zenuwstelsels maar een rekenkundige keuze. Zij maakt de voorwaartse berekening een reeks matrixvermenigvuldigingen en de gradiënt een nette toepassing van de kettingregel. Daar staan twee aannames tegenover die zelden expliciet worden gemaakt. Ten eerste kost elke berekening evenveel stappen: de weg van invoer naar uitvoer is voor elk signaal precies zo lang als het netwerk diep is. Ten tweede stroomt informatie maar één kant op.

Een biologisch zenuwstelsel doet geen van beide. Een reflexboog loopt in twee synapsen van zintuig naar spier, terwijl een afweging tientallen stappen door de cortex maakt; terugkoppeling is er eerder regel dan uitzondering. Dat verschil is niet alleen anatomisch interessant. Het bepaalt welke oplossingen een netwerk überhaupt kan vinden: in een gelaagd netwerk bestaat "sneller reageren dan nadenken" niet als optie.

ANG laat de laagstructuur los. Wat overblijft is een gerichte graaf met getypeerde knopen, waarin de enige harde eis is dat elke invoer ergens naartoe gaat en elke uitvoer ergens vandaan komt. Alles daarbinnen — hoeveel verbindingen een neuron heeft, hoe lang het pad van zintuig naar knop is, of er lussen zijn, en zelfs welke soort een neuron is — is uitkomst van het leerproces in plaats van invoer ervan.

1.2 Wat wij willen kunnen aflezen

De vraag achter dit werk is niet in de eerste plaats hoe goed het model presteert, maar welke structuur er ontstaat. Dat komt voort uit een praktisch probleem: van een te klein of slecht getraind netwerk wil je kunnen zien wáárom het faalt, niet alleen dát het faalt. Elk onderdeel van ANG is daarom zo gekozen dat het achteraf afleesbaar is. Hoeveel stappen heeft een signaal nodig van zintuig naar knop? Hoeveel terugkoppelingen zijn er gevormd? Welke van de zintuigen worden feitelijk genegeerd? Welke neuronen doen niet mee? Sectie 6 formaliseert die maten.

1.3 Bijdragen

- Een **getypeerd grafenmodel** met formele toelaatbaarheidsregels per soort, waaronder een reflexsoort die de klassieke boog zintuig $\rightarrow$ sensorisch neuron $\rightarrow$ actuator afdwingt, en een geheugensoort met een verplichte zelfverbinding.
- Een **leerregel zonder backpropagation** die werkt in aanwezigheid van cykels en discrete, gelote acties: een exacte score-functie voor de uitvoerknopen, node-perturbatie voor de verborgen knopen, en een lopende basislijn die voorkomt dat een altijd-positieve beloning simpelweg alles versterkt.
- **Structurele plasticiteit inclusief soorttoewijzing**, geformuleerd als een kostenminimalisatie over de toegestane soorten: een neuron krijgt de soort die het, gemeten aan zijn eigen bedrading, al feitelijk is.
- Een **verzameling structuurmaten** waarmee de gevormde topologie meetbaar wordt gemaakt: kortste pad, lussen via sterk samenhangende componenten, reflexbogen, deelnemende neuronen en sensorinvloed.
- Een **volledige implementatie** zonder externe afhankelijkheden, die het netwerk tijdens het leren live in beeld brengt en elke run met alle instellingen en de volledige eindstructuur wegschrijft.

1.4 Verwant werk

ANG ligt op het kruispunt van vijf onderzoeksrichtingen. Geen ervan dekt het model, maar samen bakenen zij het af.

Willekeurig bedrade netwerken

Xie et al. [14] lieten zien dat netwerken waarvan de bedrading door een grafengenerator wordt bepaald — Erdős–Rényi, Watts–Strogatz, Barabási–Albert — op ImageNet kunnen wedijveren met met de hand ontworpen architecturen; You et al. [15] onderzochten systematisch welke graafeigenschappen met prestatie samenhangen. In beide gevallen wordt de graaf één keer getrokken en daarna bevroren: er worden uitsluitend gewichten getraind. In ANG blijft de graaf veranderen zolang er geleerd wordt.

Neuro-evolutie

NEAT [11] laat topologie ontstaan door selectie over een populatie, met mutaties die knopen en verbindingen toevoegen. ANG bereikt iets vergelijkbaars binnen één individu en tijdens zijn leven: er is geen populatie, geen generatie en geen selectie, alleen ervaring.

Reservoir computing

Echo-state-netwerken en liquid state machines [7, 8] gebruiken een vaste, willekeurige recurrente graaf en trainen uitsluitend een uitleeslaag. ANG deelt de willekeurige recurrente graaf, maar traint alle verbindingen en laat de graaf zelf veranderen.

Leerregels zonder backpropagation

De leerregel combineert de score-functieschatter van REINFORCE [13] voor de discrete acties met node-perturbatie [3, 10] voor de verborgen knopen, in een vorm met eligibility traces die aansluit bij reward-gemoduleerde plasticiteit [4, 6] en bij TD(λ) [12]. In de terminologie van die literatuur is het resultaat een reward-gemoduleerde Hebbiaanse regel; de afleiding in 3.3 en 3.4 laat zien dat diezelfde regel tegelijk een

gradiëntschatter is. De dichtheidsbeloning in sectie 5 is potentiaal-gebaseerd in de zin van Ng, Harada en Russell [9], zodat zij het optimale beleid niet verandert.

Structurele plasticiteit

In de neurowetenschap is het onderscheid tussen synaptische en structurele plasticiteit gangbaar: bij de eerste verandert het gewicht van een bestaande verbinding, bij de tweede ontstaat of verdwijnt de verbinding zelf [2, 5]. ANG doet beide, en dat is het scherpste verschil met gangbare deep learning. In neuromorfe systemen is die combinatie eerder gerealiseerd, met herbedrading naast een leerregel voor de overgebleven verbindingen [1].

En hoe verhoudt dit zich tot grafenneurale netwerken?

Een GNN rekent over een graaf die als *data* gegeven is: knopen wisselen berichten uit met hun burens volgens een vaste structuur. In ANG is de graaf geen invoer maar *toestand* — hij is onderdeel van wat er geleerd wordt, en hij is aan het einde van een training een andere dan aan het begin. Dat is geen gradueel maar een categorisch verschil, en het is de reden dat de meetinstrumenten in sectie 6 nodig zijn: bij een GNN weet je de structuur al, bij ANG moet je haar achteraf vaststellen.

2 Het model

2.1 Graaf, knopen en soorten

Een ANG is een gerichte graaf met getypeerde knopen:

$$V = I \cup O \cup H, \quad |I| = 16, \quad |O| = 4, \quad E \subseteq V \times V \quad (1)$$

waarin I de invoerknopen zijn (in de hier beschreven instantie zestien zintuigwaarden), O de uitvoerknopen (vier knopen) en H de wolk ertussen. Elke knoop draagt een soort:

$$\kappa: V \rightarrow K, \quad K = \{\text{in, sens, work, refl, mem, neut, out}\} \quad (2)$$

De soorten in en out horen bij de vaste invoer- en uitvoerknopen. De vier soorten die de gebruiker kan inschakelen zijn sens (invoer-neuron), work (worker), refl (reflex of instinct) en mem (geheugen). De soort neut (neutraal) is een tijdelijke toestand van een pas gegroeid of losgeraakt neuron dat nog geen soort heeft gekregen; sectie 4.4 beschrijft hoe die toewijzing verloopt.

2.2 Toelaatbaarheid

Niet elke verbinding mag. Een relatie L over soortenparen bepaalt welke bogen bestaan mogen:

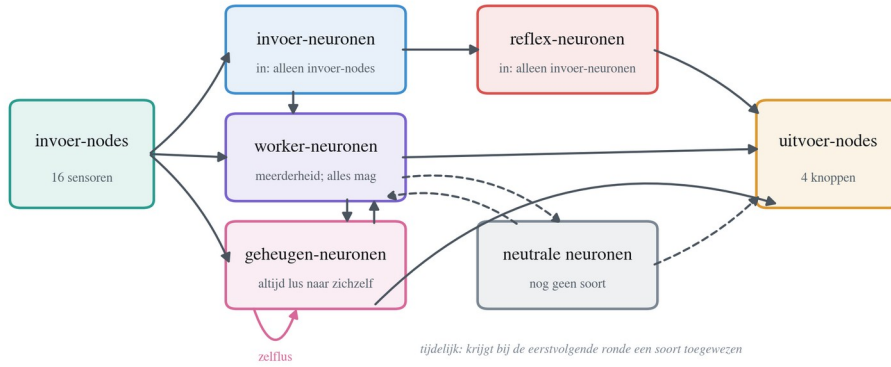
$$(a, b) \in E \Rightarrow L(\kappa(a), \kappa(b)) = 1 \quad \text{en} \quad a = b \Rightarrow \kappa(a) = \text{mem} \quad (3)$$

De regels luiden, in volgorde van toepassing:

- Naar een invoerknoop gaat nooit een verbinding toe; uit een uitvoerknoop komt nooit een verbinding vandaan.
- Een invoer-neuron accepteert als ingang uitsluitend invoerknopen. Zijn uitgang is vrij.
- Een reflex-neuron accepteert als ingang uitsluitend invoer-neuronen en heeft als uitgang uitsluitend uitvoerknopen. Er is voor deze soort geen andere mogelijkheid.
- Een invoerknoop mag niet rechtstreeks aan een uitvoerknoop hangen: elk signaal passeert minstens één neuron.
- Een zelfverbinding is uitsluitend toegestaan voor een geheugen-neuron, en is voor die soort verplicht.
- Worker-, geheugen- en neutrale neuronnen mogen voor het overige alles met alles.

De reflexregel is bewust de meest beperkende. Zij dwingt de klassieke reflexboog af: *receptor* → *sensorisch neuron* → *interneuron* → *effector*. Er is voor deze soort geen andere mogelijkheid, en dat maakt achteraf meetbaar of een brein zo'n boog heeft gevormd of alles door de wolk laat lopen.

Daarbij hoort een waarschuwing die wij pas bij het meten hebben ontdekt. Standaard mag een worker-neuron *wel* rechtstreeks aan een invoerknoop hangen. Het pad invoer → worker → knop is dan twee bogen, terwijl de reflexboog er drie telt — de reflex is dus niet de kortste maar juist de langere weg, en verliest daardoor systematisch de concurrentie om de motoraansturing. Wie wil dat de reflexboog werkelijk het kortste pad is, zet de optie aan die invoerknoppen uitsluitend aan invoer-neuronen laat hangen; dan is elk pad drie bogen lang. Die keuze heeft een prijs: alle zestien zintuigen moeten dan door een handvol invoer-neuronen, en dat is een informatie-flessenhals die het leren aanzienlijk moeilijker maakt. Beide varianten zijn instelbaar, en het verschil ertussen is een van de metingen die sectie 10 voorstelt.



Figuur 1 — De toegestane verbindingen tussen de knoopsoorten. Streepjeslijnen gelden zolang een neuron neutraal is. Er gaat nooit iets naar een invoerknoop toe, nooit iets uit een uitvoerknoop vandaan, en een invoerknoop hangt nooit rechtstreeks aan een knop.

2.3 Invarianten

Vijf eigenschappen gelden op elk moment, en worden na elke herstructurering hersteld:

- **I1** elke invoerknoop heeft uitgraad ≥ 1 ;
- **I2** elke uitvoerknoop heeft ingraad ≥ 1 ;
- **I3** elke boog is toelaatbaar volgens L ;
- **I4** elk geheugen-neuron heeft zijn zelfverbinding;
- **I5** de worker-neuronen vormen de meerderheid van de toegewezen neuronen, en elke soort blijft binnen de door de gebruiker opgegeven onder- en bovengrens (vergelijking 28).

I1 en I2 zijn de enige eis die aan de bedrading wordt gesteld; er is geen minimum of maximum aantal verbindingen per neuron. I5 is een ontwerp-eis: de wolk moet in meerderheid uit neuronen bestaan die alles mogen, zodat de gespecialiseerde soorten uitzonderingen blijven en niet de structuur gaan dicteren.

2.4 Toestandsdynamiek

De toestand van het netwerk op tijdstip t is de vector van activaties x . Eén spelstap bestaat uit P propagatiestappen. Bij elke propagatiestap wordt eerst voor elke niet-invoerknoop de netto-invoer bepaald,

$$u_j^{(s)} = b_j + \sum_{i: (i,j) \in E} w_{ij} x_i^{(s-1)} \quad (4)$$

en vervolgens de nieuwe activatie:

$$x_j^{(s)} = \tanh(u_j^{(s)} + \xi_j), \quad \xi_j \sim \mathcal{U}(-\sigma_h, \sigma_h) \quad (5)$$

met ξ een uniforme exploratieruis. Geheugen-neuronen zijn lekkende integratoren, zodat zij hun waarde over de tijd vasthouden:

$$x_j^{(s)} = (1 - \alpha)x_j^{(s-1)} + \alpha \tanh(u_j^{(s)} + \xi_j) \quad (\kappa(j) = \text{mem}) \quad (6)$$

De actualisering is synchroon: alle knopen lezen de toestand van de vorige propagatiestap. Daardoor is een cyclus geen probleem — een zelfverbinding of een terugkoppeling door de wolk is gewoon een bijdrage aan de netto-invoer van de volgende stap. De parameter P bepaalt hoe ver een signaal binnen één spelstap door de wolk reist. Bij $P = 1$ kost elke boog een tijdstap, en is een korte reflexboog daadwerkelijk sneller dan een lange omweg; bij hogere P vervaagt dat verschil.

2.5 Actieselectie

De vier uitvoerknopen zijn geen deterministische drempels maar kansen. Dat is geen esthetische keuze: het maakt het beleid differentieerbaar in zijn parameters en levert daarmee een exacte gradiënt (sectie 3.3).

$$p_k = \sigma\left(\frac{u_k}{\tau}\right), \quad a_k \sim \text{Bernoulli}(p_k), \quad k \in \mathcal{O} \quad (7)$$

waarbij τ een temperatuur is die de scherpte van de beslissing regelt. De vier geloten bits worden vertaald naar een verplaatsing; twee tegelijk levert een diagonaal, alle vier of geen enkele levert stilstand:

$$\Delta \mathbf{q}_t = v \cdot \frac{(a_r - a_l, a_d - a_u)}{\|(a_r - a_l, a_d - a_u)\|} \quad (8)$$

Bij evaluatie zonder leren vervalt de loting en geldt $a_k = 1$ dan en slechts dan als $p_k > 1/2$, wat onafhankelijk is van τ .

3 Leren zonder backpropagation

3.1 Waarom backpropagation hier niet past

Drie eigenschappen van ANG sluiten de gebruikelijke aanpak uit.

- **De graaf bevat cykels.** Backpropagation vereist een acyclische rekenalgoritme. Voor recurrente netwerken bestaat backpropagation-through-time, maar dat vraagt om het uitrollen van de volledige tijdreeks en om een topologie die tijdens die reeks niet verandert.
- **De topologie verandert tijdens het leren.** Verbindingen verdwijnen, komen erbij, en neuronen veranderen van soort. Een vaste rekenalgoritme waarop een gradiënt gedefinieerd kan worden, bestaat dus niet.
- **De acties zijn discreet en geloot.** Er is geen doellabel om een fout tegen af te zetten, alleen een beloningssignaal, en de stap van kans naar knopdruk is niet differentieerbaar.

Wat wél kan, is een schatter van de gradiënt van de verwachte opbrengst opbouwen uit grootheden die lokaal aan elke verbinding beschikbaar zijn. Dat is de weg die hier gevolgd wordt.

3.2 Doelfunctie

Wij maximaliseren de verwachte som van beloningen over een poging:

$$J(\theta) = \mathbb{E}_{\pi_\theta} \left[\sum_{t=0}^T r_t \right], \quad \theta = \{w_{ij}, b_j\} \quad (9)$$

3.3 De uitvoerknopen: een exacte score-functie

Omdat de knoppen Bernoulli-variabelen zijn met kans $p_k = \sigma(u_k/\tau)$, is de afgeleide van de log-waarschijnlijkheid van de gekozen actie naar de netto-invoer exact bekend:

$$\frac{\partial}{\partial u_k} \log \pi_\theta(a_k) = a_k - p_k \quad (10)$$

en daarmee, via de kettingregel over één enkele stap, naar het gewicht van een inkomende verbinding:

$$\frac{\partial}{\partial w_{ik}} \log \pi_\theta(a_k) = \frac{1}{\tau} \tilde{x}_i (a_k - p_k) \quad (11)$$

Dit is de klassieke score-functieschatter [12]. De factor $1/\tau$ is een constante schaalfactor die in de leersnelheid wordt opgenomen. Het aantrekkelijke is dat de term $(a_k - p_k)$ precies meet *hoezeer de genomen actie afweek van wat het netwerk gemiddeld zou doen* — en alleen die afwijking is iets waaraan beloning toegeschreven mag worden.

3.4 De wolk: node-perturbatie

Voor de verborgen knopen bestaat zo'n exacte uitdrukking niet. Wij gebruiken node-perturbatie [10, 3]. Elke spelstap wordt twee keer doorgerekend vanaf dezelfde begintoestand: één keer zonder ruis en één keer met ruis. Het verschil is per definitie de duw die de exploratie gaf:

$$\delta_j = x_j - \bar{x}_j \quad (12)$$

Omdat de ruis symmetrisch is, geldt $E[\delta] = 0$, en is het product van de activatie vóór de verbinding met de perturbatie erna, gewogen met de beloningsafwijking, een schatter van de gradiënt:

$$E[\tilde{x}_i \delta_j \hat{A}] = c_j \frac{\partial J}{\partial w_{ij}} + \mathcal{O}(\sigma_h^2), \quad c_j > 0 \quad (13)$$

De evenredigheidsconstante is niet één getal voor het hele netwerk. Voor een knoop in de wolk komt zij uit de variantie van de perturbatie en de helling van de overdrachtsfunctie; voor een knop is zij de schaal van de exacte score-functie uit 3.3:

$$c_j \approx \text{Var}(\xi) \varphi'(u_j) \quad (j \in \mathcal{H}), \quad c_k = \frac{1}{\tau} \quad (k \in \mathcal{O}), \quad \text{Var}(\xi) = \frac{\sigma_h^2}{3} \quad (37)$$

Dat onderscheid is niet cosmetisch. $\text{Var}(\xi)$ is bij de standaardinstellingen ongeveer $3,7 \cdot 10^{-3}$, dus de twee delen van de update verschillen van nature ruwweg twee ordes in schaal. In de gebruikelijke formulering van node-perturbatie [10, 3] wordt daarvoor gecorrigeerd met een factor $1/\text{Var}(\xi)$; ANG deed dat niet, en sectie 3.11 laat zien wat dat kost. De schatter is verder zuiver op een term van tweede orde in de ruissterkte na. Belangrijker voor de praktijk is dat hij volledig lokaal is: hij vraagt alleen de activatie aan de ene kant van een verbinding en de perturbatie aan de andere. Er is geen pad terug door het netwerk nodig, en dus ook geen acyclische structuur.

3.5 Eligibility traces

Een beloning valt zelden op hetzelfde moment als de actie die haar veroorzaakte. Elke verbinding houdt daarom een spoor bij van haar recente bijdrage:

$$e_{ij}(t) = \lambda e_{ij}(t-1) + (1-\lambda) \tilde{x}_i(t) \delta_j(t), \quad \tilde{x}_i(t) \equiv x_i^{(P-1)}(t) \quad (14)$$

Welke activatie hier aan de presynaptische kant hoort, ligt minder voor de hand dan het lijkt. Een tik bestaat uit P propagatiestappen (vgl. 4), en die worden synchroon uitgevoerd: eerst worden alle netto-invoeren uit de bestaande toestand berekend, pas daarna wordt geschreven. De uitvoer van knoop j in deze tik is dus voortgebracht door de toestand aan het *begin* van de laatste propagatiestap, niet door de toestand die erna in het netwerk staat. Dat is de grootheid $\tilde{x}_i$ in vergelijking 14, en om dezelfde reden ook in 11 en 13. Voor invoerknopen maakt het niets uit — die worden aan het begin van de tik op hun sensorwaarde gezet en veranderen daarna niet meer — maar voor elke verbinding die uit de wolk zelf vertrekt wél, en bij $P > 1$ groeit het verschil.

De normalisatie met $(1 - \lambda)$ is niet cosmetisch: zonder die factor groeit het spoor met $1/(1 - \lambda)$, waardoor een verandering van λ ongemerkt ook de effectieve leersnelheid verandert. Met de factor erbij blijft de schaal van het spoor gelijk en regelt λ uitsluitend hoe ver terug de beloning wordt uitgesmeerd. Voor de biasterm geldt dezelfde regel met de pre-activatie gelijk aan één.

Een implementatiedetail dat er wél toe doet

Tot 9 september 2026 stond in de implementatie de activatie ná de propagatie op de plaats van $\bar{x}_i$. De leerregel die daadwerkelijk draaide was daarmee, voor elke verbinding die uit de wolk vertrok, één propagatiestap uit de pas met de regel die hierboven beschreven staat. Wij melden dit niet uit volledigheidscrang: het is precies de term waarop de aanspraak van sectie 3.4 rust, en een lezer die de code naast de formules legt hoort geen verschil te vinden.

maat	oude term	juiste term	verschil (gepaard)	Mann-Whitney
succes over de laatste 20 pogingen	85.0% $\pm$ 5.9	89.2% $\pm$ 5.6	+4.2 $\pm$ 7.0 pp	p = 0.326
toets op onbekende werelden	68.3% $\pm$ 3.0	67.9% $\pm$ 4.3	-0.4 $\pm$ 3.9 pp	p = 0.931
succes over de hele training	63.3% $\pm$ 2.8	63.5% $\pm$ 1.9	+0.2 $\pm$ 2.2 pp	p = 0.954

Waarom het verschil in de praktijk klein blijft, bleek pas bij de controle van sectie 3.11: over een losse tik met een willekeurige wolktoestand schelen de twee sporen tientallen procenten, maar in het beloningsgewogen gemiddelde over een hele poging is het verschil 0.7 %. De toestand van de wolk verandert langzaam, dus de activatie van vóór en ná één propagatiestap lijken sterk op elkaar.

De correctie is gemeten en niet aangenomen: 12 breinzaden per conditie, 500 pogingen per run, verder identieke instellingen en dezelfde wereldzaden. Het verschil op de toets valt binnen de spreiding tussen zaden: op deze taak leerde het netwerk ook met de foutieve term, en de correctie levert geen aantoonbare winst in prestatie op. Dat maakt haar niet minder nodig — de formule en de code beschrijven nu hetzelfde algoritme, en de numerieke controle van sectie 3.4 is pas zinvol als dat zo is — maar het is geen resultaat dat wij als verbetering mogen presenteren. Beide reeksen staan volledig in experimenten/runs.csv onder de condities trace-oud en trace-nieuw.

3.6 Basislijn en advantage

Beloning in absolute zin is geen bruikbaar leersignaal. Beweegt de agent consequent in de goede richting, dan is elke beloning positief en zou elke verbinding die actief was versterkt worden — inclusief de verbindingen die niets bijdroegen. Wij trekken daarom een lopende basislijn af:

$$\bar{r}_t = \bar{r}_{t-1} + \beta(r_t - \bar{r}_{t-1}), \quad \beta = 0.02 \quad (15)$$

$$\hat{A}_t = \text{clip}(r_t - \bar{r}_t, -c, c), \quad c = 10 \quad (16)$$

$\hat{A}_t$ is een advantage-schatting: hoeveel beter of slechter deze stap uitpakte dan het brein de laatste tijd gewend was. De begrenzing voorkomt dat één uitschieter — bijvoorbeeld het bereiken van het doel — de gewichten in één keer omgooit. In de ontwikkeling van dit model bleek het weglaten van de basislijn de belangrijkste oorzaak van falen: zonder haar liepen alle gewichten binnen enkele honderden stappen tegen hun plafond aan.

3.7 De begrensde update

De ruwe verandering van een gewicht is het product van leersnelheid, advantage en spoor:

$$\widetilde{\Delta w}_{ij} = \eta_t \hat{A}_t e_{ij} \quad (17)$$

Daarop werken drie begrenzingen, die elk een ontwerpeis afdwingen.

Niet oneindig versterken

Een verandering die een gewicht verder van nul af duwt, wordt gedempt naarmate dat gewicht zijn plafond nadert; een verandering die het naar nul toe brengt niet:

$$\varphi(w, \tilde{\Delta}) = 1 - \frac{|w|}{w_{\max}} \cdot \mathbf{1}[\text{sgn}(\tilde{\Delta}) = \text{sgn}(w)] \quad (18)$$

Niet abrupt

De verandering per stap is hard begrensd, en het gewicht zelf blijft binnen zijn bereik:

$$\Delta w_{ij} = \text{clip}(\varphi \cdot \tilde{\Delta} w_{ij}, -\Delta_{\max}, \Delta_{\max}) \quad (19)$$

$$w_{ij} \leftarrow \text{clip}(w_{ij} + \Delta w_{ij}, -w_{\max}, w_{\max}) \quad (20)$$

De combinatie is belangrijk. De damping alleen zou een gewicht asymptotisch tegen het plafond aan laten kruipen zonder het ooit te bereiken, maar zegt niets over de snelheid; de stapbegrenzing alleen zou een gewicht in een rechte lijn naar zijn plafond laten lopen. Samen leveren zij een verbinding die geleidelijk sterker wordt en daarbij steeds moeilijker nog sterker te maken is.

3.8 Vervaging

Eenmaal per poging zakt elk gewicht een klein stuk terug naar nul:

$$w_{ij} \leftarrow (1 - \rho) w_{ij} \quad (21)$$

Wat niet regelmatig bekrachtigd wordt, verdwijnt daarmee vanzelf, en wordt uiteindelijk gesnoeid (sectie 4.1). De keuze om dit per poging te doen en niet per spelstap is empirisch: bij een paar honderd stappen per poging overheerst een vervaging per stap het leersignaal volledig, en krimpen de gewichten monotoon ongeacht wat het netwerk presteert.

3.9 Exploratieschema

De exploratie loopt gedurende de training terug, maar nooit tot nul:

$$\varepsilon_e = \varepsilon_0 + (\varepsilon_{\infty} - \varepsilon_0) \min\left(1, \frac{e}{E}\right), \quad \varepsilon_{\infty} = \max(0.3 \varepsilon_0, 0.03) \quad (22)$$

Uit die ene grootte volgen de drie plekken waar exploratie in het model binnenkomt:

$$\eta_e = \eta_0 \frac{\varepsilon_e}{\varepsilon_0}, \quad \tau_e = 0.40 + 2 \varepsilon_e, \quad \sigma_h = 0.35 \varepsilon_e \quad (23)$$

De ondergrens is essentieel. Zonder exploratie is δ gelijk aan nul, valt het leersignaal weg, terwijl de vervaging en het snoeien doorgaan — een goed getraind netwerk valt dan aan het einde van de training alsnog uit elkaar. Het meeschalen van de leersnelheid met de exploratie houdt de verhouding tussen signaal en aanpassing constant.

3.10 Het algoritme in zijn geheel

Algoritme 1 — leren

```
voor elke poging e = 1 ... E:
   $\varepsilon \leftarrow \text{exploratieschema}(e)$  (22)
  zet de toestand van het netwerk en alle sporen op nul
  voor elke spelstap t = 1 ... T:
    neem waar  $\rightarrow$  invoervector s
    schone doorrekening  $\tilde{x} \leftarrow \text{propagate}(s, \text{ruis} = 0)$ 
    ruisige doorrekening  $x \leftarrow \text{propagate}(s, \text{ruis} = \varepsilon)$ ; bewaar  $\tilde{x}$  = toestand vóór de
  laatste stap
   $\delta \leftarrow x - \tilde{x}$  (12)
  voor elke knop k:  $a_k \sim \text{Bernoulli}(p_k)$ ;  $\delta_k \leftarrow a_k - p_k$  (7, 10)
```

```

voer de actie uit → beloning r

$$e_{ij} \leftarrow \lambda e_{ij} + (1-\lambda) \tilde{x}_i \delta_j \quad (14)$$


$$\tilde{A} \leftarrow \text{clip}(r - \tilde{r}, -c, c); \quad \tilde{r} \leftarrow \tilde{r} + \beta(r - \tilde{r}) \quad (15, 16)$$


$$w_{ij} \leftarrow \text{clip}(w_{ij} + \text{clip}(\varphi \cdot \eta \tilde{A} e_{ij}, \pm \Delta_{\max}), \pm w_{\max}) \quad (17-20)$$


$$w \leftarrow (1 - \rho) w \quad (21)$$

als e mod K = 0: herstructureer (algoritme 2)

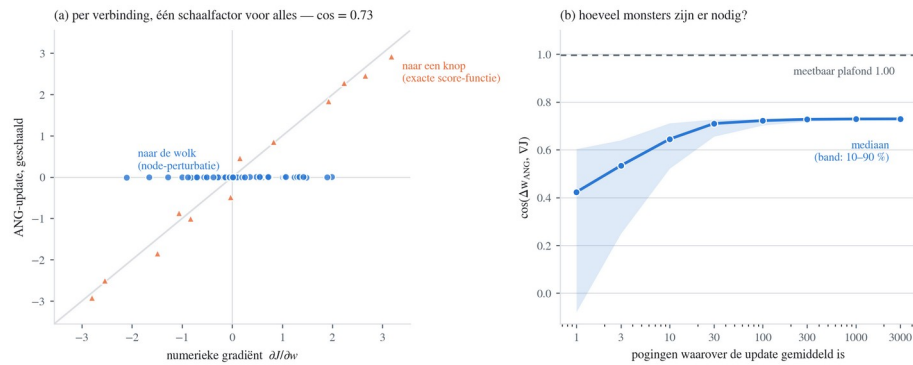
```

3.11 Numerieke controle van de leerregel

De aanspraak van 3.4 — dat de update een schatter van de gradiënt is — was tot nu toe een afleiding en geen meting. Wij toetsen haar op een miniatuur-ANG met vier verborgen knopen en een bevroren topologie, waar de gradiënt nog met de hand na te rekenen is:

$\partial J / \partial w_{ij} \approx (J(w+\varepsilon) - J(w-\varepsilon)) / 2\varepsilon$, met J geschat als de gemiddelde totale beloning over een vast blok pogingen.

Twee dingen maken die schatting bruikbaar. Ten eerste **gemeenschappelijke toevalsgetallen**: elke poging heeft een vaste eigen toevalsgenerator en een vaste wereld, identiek voor elke waarde van w . Zonder die truc verdrinkt het verschil tussen $J(w+\varepsilon)$ en $J(w-\varepsilon)$ in de ruis van het beleid zelf. Ten tweede een **ijking van de meetlat**: de numerieke gradiënt wordt twee keer berekend op onafhankelijke blokken pogingen. De cosinus tussen die twee helften zegt hoeveel van de gemeten richting signaal is, en dus hoe hoog een cosinus überhaupt kan worden. Bij 800 pogingen per helft is die 0.987; het meetbare plafond ligt daarmee op 1.00. Een cosinus die daar tegenaan zit, is niet meer van een perfecte uitlijning te onderscheiden.



Figuur 2 — De ANG-update tegen de numerieke gradiënt op de miniatuurgraaf. Links: per verbinding, met één gemeenschappelijke schaalfactor. De verbindingen naar de knoppen liggen op de diagonaal; de verbindingen naar de wolk liggen plat tegen de nullijn — hun richting klopt wel, hun grootte niet. Rechts: de cosinus als functie van het aantal pogingen waarover de update gemiddeld wordt.

Meetpunt: begin

deel van de update	n	cos met ∇J	schaalfactor c	aandeel in $ \Delta w $	aandeel in $ \nabla J $
naar een knop (score-functie)	12	0.99	2	100 %	73 %
naar de wolk (node-perturbatie)	62	0.27	96	1 %	68 %

Over alle verbindingen samen is de cosinus 0.73; schaalt men de twee delen apart, dan wordt zij 0.75. De schaalfactor van het wolksdeel is 44× die van het knopdeel.

Meetpunt: half getraind

deel van de update	n	cos met ∇J	schaalfactor c	aandeel in $ \Delta w $	aandeel in $ \nabla J $
naar een knop (score-functie)	12	0.85	2	100 %	12 %
naar de wolk (node-perturbatie)	62	0.83	608	2 %	99 %

Over alle verbindingen samen is de cosinus 0.12; schaalt men de twee delen apart, dan wordt zij 0.83. De schaalfactor van het wolksdeel is 349× die van het knopdeel.

Meetpunt: begin $\text{prop}=2$

deel van de update	n	cos met ∇J	schaalfactor c	aandeel in $ \Delta w $	aandeel in $ \nabla J $
naar een knop (score-functie)	12	1.00	2	100 %	71 %
naar de wolk (node-perturbatie)	62	0.36	225	0 %	70 %

Over alle verbindingen samen is de cosinus 0.71; schaalt men de twee delen apart, dan wordt zij 0.75. De schaaftactor van het wolkeel is $105\times$ die van het knopdeel.

Wat de meting zegt

- **De exacte score-functie klopt.** Voor de verbindingen naar de knoppen is de cosinus 0.99 / 0.85 / 1.00 op de drie meetpunten. Bij een meetbaar plafond van 1.00 / 0.99 / 0.99 is dat niet van exact te onderscheiden: vergelijking 11 doet wat zij belooft.
- **Node-perturbatie wijst de goede kant op, maar ruw.** Voor de verbindingen naar de wolk is de cosinus 0.27 / 0.83 / 0.36. Positief en bruikbaar, maar ver van exact — en sterk afhankelijk van het punt in de gewichtsruimte.
- **De twee delen staan niet op dezelfde schaal.** Dit is de belangrijkste uitkomst. Het wolkeel van de update is gemiddeld ongeveer $166\times$ te klein ten opzichte van het knopdeel, wat overeenkomt met de ontbrekende normalisatie $1/\text{Var}(\xi)$ uit vergelijking 37 (≈ 272 bij deze exploratie). Het gevolg is te zien in figuur 2a: met één leersnelheid krijgt de wolk feitelijk nauwelijks een update, terwijl de echte gradiënt daar 68 % / 99 % / 70 % van zijn lengte heeft liggen.
- **De ruis is niet het probleem.** Uit figuur 2b: het gemiddelde over ongeveer dertig pogingen zit al op de eindwaarde. Meer monsters helpen daarna niet meer — wat er dan nog tussen zit is geen variantie maar vertekening.

De keuze van ε stuurt de uitkomst niet: over $\varepsilon = 0.01$ tot 0.12 blijft de cosinus met de schatter tussen 0.69 en 0.72.

Wat de correctie waard is — nog niets, zonder de rest bij te stellen

De ontbrekende factor toevoegen is één regel code. Wij hebben het gedaan, op het volledige brein, met g als versterking van het wolkeel van het spoor — en het resultaat is de moeite waard om te melden juist omdat het tegenvalt:

versterking g	succes laatste 20	toets	actieve verbindingen
1 (huidige leerregel)	90.0% $\pm$ 8.1	67.5% $\pm$ 5.6	1441
10	48.1% $\pm$ 4.5	21.3% $\pm$ 6.9	308
30	34.4% $\pm$ 5.4	9.4% $\pm$ 3.4	324
100	40.0% $\pm$ 1.9	5.6% $\pm$ 3.4	520
272	36.9% $\pm$ 4.5	1.9% $\pm$ 2.6	1198

8 zaden per waarde, 500 pogingen, leersnelheid ongewijzigd. Het leren stort in. Dat is geen weerlegging van de meting maar een bevestiging van wat zij zegt: als het wolkeel twee ordes groter wordt terwijl η en de stapbegrenzing op de oude schaal blijven staan, loopt elke update in de wolk tegen de begrenzing uit sectie 3.7 aan en verliest die haar functie. De huidige η is stilzwijgend op de ónjuiste schaal afgesteld.

De schone tegenproef schaalt de leersnelheid mee met $1/g$. Dan blijft het wolkeel op zijn oude grootte en wordt het knopdeel g keer zachter — dezelfde verschuiving in de onderlinge verhouding, zonder dat de totale stap ontploft:

versterking g	succes laatste 20	toets	actieve verbindingen
1 (huidige leerregel)	90.0% $\pm$ 8.1	67.5% $\pm$ 5.6	1441
10	52.5% $\pm$ 8.7	37.5% $\pm$ 6.4	1475
30	15.0% $\pm$ 11.9	14.4% $\pm$ 8.2	1478

versterking g	succes laatste 20	toets	actieve verbindingen
100	3.8% ± 6.1	4.4% ± 5.1	1487
272	0.0% ± 0.0	2.5% ± 3.7	1509

Ook zo wint geen enkele waarde van g het van de bestaande verhouding. De conclusie is dan dat de scheve schaal wel aantoonbaar is op de miniatuurgraaf, maar dat het herstellen ervan met alleen deze ene knop niet werkt: leersnelheid, stapbegrenzing en exploratie hangen samen en moeten als geheel opnieuw afgesteld worden. Dat is een ablatie op zichzelf en hoort niet in deze sectie thuis.

Wat er wél staat is dit: de leerregel bevat een aantoonbare schaalfout, die op de miniatuurgraaf de cosinus met de echte gradiënt van 0,12 naar 0,83 scheelt, en die in het volledige model niet met één parameter recht te zetten is. Dat is een scherpere en eerlijkere uitspraak dan de oorspronkelijke "de schatter is zuiver op een tweede-ordeterm na".

4 Structurele plasticiteit

Tot hier verandert alleen de sterkte van bestaande verbindingen. De tweede helft van het model verandert de graaf zelf. Dat gebeurt niet elke stap maar in rondes, eens per K pogingen, zodat de structuur rustiger verandert dan de gewichten.

4.1 Snoeien

Elke verbinding houdt bij hoe lang zij al zwak is. Eén ronde zwak zijn is niet genoeg; pas wie twee rondes achtereen onder de drempel blijft, verdwijnt:

$$n_{ij} \leftarrow (n_{ij} + 1) \cdot \mathbf{1}[|w_{ij}| < \theta_p], \quad \text{snoei indien } n_{ij} \geq 2 \quad (24)$$

Een zelfverbinding van een geheugen-neuron wordt nooit gesnoeid (invariant I4), en een verbinding die de laatste uitweg van een invoerknoop of de laatste toevoer van een uitvoerknoop is evenmin (I1, I2).

4.2 Aangroei van verbindingen

In dezelfde ronde komen er nieuwe verbindingen bij, met een klein willekeurig gewicht. De bron wordt gekozen met een lichte voorkeur voor magere neuronen: uit drie willekeurige kandidaten wordt die met de laagste graad genomen. Zo groeit de bedrading eerder aan waar weinig zit dan waar al veel zit, zonder dat er een harde regel over graden nodig is. Elke voorgestelde boog wordt getoetst aan L en genegeerd als hij niet mag.

4.3 Groei van neuronen

Als het leren vastloopt, mag er materiaal bij. Stagnatie wordt gemeten door de gemiddelde opbrengst van het laatste blok pogingen te vergelijken met dat daarvóór:

$$\text{stagnatie} \Leftrightarrow \frac{1}{k} \sum_{e=E_0-k}^{E_0-1} R_e \leq 1.02 \cdot \frac{1}{k} \sum_{e=E_0-2k}^{E_0-k-1} R_e + 0.05 \quad (25)$$

Is aan die voorwaarde voldaan en is het plafond op het aantal neuronen nog niet bereikt, dan komen er nieuwe neuronen bij. Zij komen neutraal binnen: zij horen nog bij geen enkele soort en mogen daardoor alles. Twee details bepalen of zij ooit meedoen. Ten eerste worden zij geboren met ongeveer dezelfde graad als de rest van de wolk, niet met een handvol verbindingen: in een netwerk met duizenden verbindingen valt een neuron met vier bogen volledig weg in de ruis. Ten tweede krijgen hun verbindingen een respijt van twee opruimrondes, zodat zij niet gesnoeid worden voordat zij zich hebben kunnen bewijzen. Daarbovenop krijgt elk nieuw neuron gegarandeerd één boog vanaf een invoerknoop en één naar een uitvoerknoop, zodat het vanaf het begin in een pad ligt en dus leersignaal ontvangt.

Dat dit ertoe doet is geen theorie. Met een klein vast aantal startverbindingen bleef in onze metingen bijna de helft van de neuronen na afloop losgekoppeld — het waren vrijwel allemaal bijgegroeide neuronen die nooit geïntegreerd raakten. Met graadgelijke geboorte en het respijt daalde dat aandeel naar enkele procenten, zonder dat de prestatie veranderde. De maat *deelnemende neuronen* uit sectie 6 was precies het instrument dat dit zichtbaar maakte.

4.4 Soorttoewijzing: de meest specifieke soort die past

De vraag welke soort een neuron moet krijgen, beantwoorden wij niet met een aparte heuristiek per soort maar met één criterium: **welke soort is dit neuron, gemeten aan de bedrading die het al heeft?** De kosten van een soort zijn de gewichtsmassa die verloren zou gaan omdat de regels van die soort haar verbieden:

$$C(i, t) = \sum_{(a, b) \in E_i} \mathbf{1}[\neg L_t(a, b)](|w_{ab}| + 0.02) + 0.05 \cdot \mathbf{1}[t \neq \text{work}] \quad (26)$$

waarin E_i de verbindingen zijn waarbij neuron i betrokken is en L_t de toelaatbaarheidsrelatie zoals die zou gelden als i soort t had. De kleine constante per verboden boog voorkomt dat het schrappen van louter nulverbindingen gratis is.

Op kosten alleen kan echter niet gekozen worden, en dat is een les die ons een halve dag heeft gekost. **Blijven kost per definitie niets** — alle bestaande verbindingen zijn immers toelaatbaar — en omschakelen naar worker kost ook niets, want die soort verbiedt niets. Een criterium dat de goedkoopste soort kiest, verandert dus vroeg of laat elk neuron in een worker, en de gespecialiseerde soorten verdwijnen stilletjes uit het netwerk. In onze metingen gebeurde precies dat: van negen reflex-neuronen waren er na vierhonderd pogingen nog drie over, en van de werkende reflexbogen bleef er geen enkele staan.

Wat ontbreekt is een tegenkracht: hoe strenger de regels van een soort, hoe meer het zegt dat een neuron er toch aan voldoet. Wij geven elke soort daarom een *specificiteitsbonus* — reflex het hoogst, dan invoer-neuron, dan geheugen, en worker nul — en kiezen de laagste som:

$$T(i) = \operatorname{argmin}_{t \in A(i)} C(i, t) \quad (27)$$

over de toegestane verzameling $A(i)$, die de quota van de gebruiker en de meerderheidsregel afdwingt:

$$n_{\text{work}} > n_{\text{sens}} + n_{\text{refl}} + n_{\text{mem}}, \quad m_t \leq n_t \leq M_t \quad (28)$$

Een neutraal neuron neemt altijd de beste toegestane soort — het heeft niets te verliezen. Een neuron dat al een soort heeft, wisselt alleen als de nieuwe soort duidelijk beter scoort dan blijven, met een marge μ die heen-en-weergeschuif voorkomt, én als de kosten onder de door de gebruiker gestelde drempel blijven:

$$T(i) \neq \kappa(i) \Leftrightarrow C(i, T(i)) - S(T(i)) < -S(\kappa(i)) - \mu \text{ en } C(i, T(i)) \leq C_{\max}, \quad \mu = 0.02 \quad (36)$$

Wat er daarna aan verboden verbindingen overblijft, wordt verwijderd, en een neuron dat geheugen wordt, krijgt zijn zelfverbinding. Het aardige van deze formulering is dat de bekende gevallen er als bijzondere gevallen uit rollen. Een neuron dat alleen invoerknopen als bron heeft, kost niets als invoer-neuron en wint dus van worker. Een neuron dat uitsluitend tussen invoer-neuronen en knoppen hangt, is per definitie een reflex en krijgt de hoogste bonus. Een neuron in een lus kost als reflex zijn hele terugkoppeling en blijft dus worker of wordt geheugen. Er is geen aparte regel per soort nodig — alleen een rangorde van strengheid.

4.5 Recycling

Een neuron dat na het snoeien geen enkele verbinding meer overhoudt, verliest zijn soort en wordt weer neutraal. Het is daarmee vrij materiaal: bij de volgende ronde kan het opnieuw bedraad worden en een andere soort krijgen. Zo verdwijnt niets uit het netwerk, maar wordt wel hergebruikt wat niet meer meedoet.

4.6 Herstel van de invarianten

Aan het einde van elke ronde worden de invarianten hersteld: elke invoerknoop zonder uitweg krijgt er een, elke uitvoerknoop zonder toevoer krijgt er een, elk geheugen-neuron zonder zelfverbinding krijgt haar terug. Pas daarna wordt de graaf weer vastgelegd in de compacte datastructuren van sectie 7.

Algoritme 2 — herstructureren

```
herstructureer(breïn B, instellingen c):  
  1 snoei zwakke verbindingen (24)  
  2 neuronen zonder verbindingen → neutraal  
  3 voor elk neutraal neuron:  $T(i) \leftarrow \operatorname{argmin} C(i, t)$  (26, 27)  
    voor enkele bestaande neuronen: idem, mits  $C \leq C_{\max}$   
    verwijder de verbindingen die daardoor verboden zijn  
  4 als stagnatie en  $n < n_{\max}$ : voeg neutrale neuronen toe (25)  
  5 laat nieuwe verbindingen aangroeien (voorkeur: lage graad)  
  6 herstel I1, I2, I4  
  7 herbereken de samenstelling en de tekenpositie van de wolk
```

5 Omgeving, waarneming en beloning

Een leerregel is niet los te beschrijven van wat er te leren valt. De instantie waarin ANG hier draait is een continue tweedimensionale wereld waarin een karakter een doel moet bereiken zonder tegen obstakels te botsen. Start en doel liggen tegen twee tegenoverliggende randen, en welke randen dat zijn wisselt per wereld, zodat een vaste voorkeursrichting geen oplossing is. De obstakels liggen in het middengebied. Elke gegenereerde wereld wordt met een breedtezoektocht op oplosbaarheid getoetst voordat hij gebruikt wordt.

5.1 Waarneming

Het netwerk krijgt zestien getallen, en verder niets — geen coördinaten, geen kaart, geen geheugen van eerdere pogingen. Acht ervan meten in acht windrichtingen de nabijheid van het dichtstbijzijnde obstakel of de wereldrand:

$$s_d = 1 - \frac{\min(\rho_d, \rho_{\max})}{\rho_{\max}}, \quad d = 1, \dots, 8 \quad (29)$$

waarin ρ_d de afstand langs straal d is. De andere acht coderen het doel als een richting-met-nabijheid: hoe beter een richting naar het doel wijst en hoe dichterbij het doel is, hoe hoger de waarde.

$$g_d = \max(0, \langle \mathbf{u}_d, \hat{\mathbf{g}} \rangle) \left(1 - \frac{\min(D, D_{\max})}{D_{\max}} \right) \quad (30)$$

Beide families zijn zo geschaald dat nul "niets aan de hand" betekent. Dat is geen detail: bij een nulwaarde levert een verbinding geen bijdrage aan de netto-invoer en ook geen spoor, zodat een neuron dat op een stille sensor is aangesloten vanzelf onbelast blijft.

5.2 Beloning

De beloning per stap is een som van vijf termen:

$$r_t = c_p(D_{t-1} - D_t) - c_s - c_i \mathbf{1}[\text{stil}] - c_b \mathbf{1}[\text{botsing}] + c_g \left(1 + \frac{1}{2} \left(1 - \frac{t}{T} \right) \right) \mathbf{1}[\text{doel}] \quad (31)$$

De eerste term belooft dichterbij het doel komen, de tweede is een kleine tijdskost, de derde bestraft stilstand, de vierde een botsing, en de vijfde is de eindbeloning met een bonus naarmate het doel sneller bereikt wordt.

5.3 De dichtheidsterm is potentiaal-gebaseerd

De term die vooruitgang beloont, verdient een opmerking. Het is verleidelijk te denken dat zo'n term het probleem verandert — dat het netwerk leert "dichter bij komen" in plaats van "het doel bereiken". Dat is hier niet zo. Met de potentiaal $\Phi(s) = -c_p D(s)$, waarin D de afstand tot het doel is, geldt namelijk

$$\Phi(s) = -c_p D(s) \quad \Rightarrow \quad c_p(D_{t-1} - D_t) = \gamma \Phi(s_t) - \Phi(s_{t-1}), \quad \gamma = 1 \quad (32)$$

en dat is precies de vorm waarvan Ng, Harada en Russell [9] hebben aangetoond dat zij het optimale beleid ongemoeid laat. De term versnelt het leren zonder de opgave te veranderen. Zet men hem op nul, dan blijft dezelfde opgave over, maar met beloning die pas aan het eind komt — een goede manier om te laten zien hoe moeizaam leren wordt bij ijl beloningssignaal.

5.4 Hoe je een stochastisch beleid toetst

Bij het toetsen op onbekende werelden ligt een fout op de loer die wij eerst zelf gemaakt hebben. Het beleid is stochastisch: elke knop is een kans en de actie wordt geloot. Het is verleidelijk om bij een toets de loting weg te laten en steeds de waarschijnlijkste knop te nemen, want dat lijkt "het geleerde beleid zonder ruis". Dat is het niet — het is een ander, en meetbaar slechter beleid. Een agent die deterministisch twee knoppen tegelijk ingedrukt houdt tegen een muur, blijft daar oneindig staan; dezelfde kansen laten hem binnen een paar stappen loskomen.

De toets draait daarom met dezelfde loting als tijdens het leren, maar zonder leren, zonder ruis in de wolk en met een eigen toevalsgenerator, zodat de toets reproduceerbaar is en de training niet verstoort. De strengere variant blijft als optie beschikbaar; het verschil tussen beide is zelf een interessante meting, want het zegt hoeveel van de prestatie op de scherpte van de beslissingen berust en hoeveel op het blijven bewegen.

6 Het meten van de gevormde structuur

Wat een ANG uiteindelijk gebouwd heeft, is de eigenlijke uitkomst. De volgende maten worden op de actieve deelgraaf berekend, dat wil zeggen op de verbindingen die zwaar genoeg zijn om ertoe te doen:

$$E_\theta = \{(a, b) \in E : |w_{ab}| > 0.08 w_{\max}\} \quad (33)$$

De drempel is bewust *relatief* aan het zwaarste gewicht dat er werkelijk is, en niet aan het plafond $w_{\max}$. Gewichten blijven in de praktijk ruim onder dat plafond; een absolute drempel verklaart daardoor bijna elke verbinding voor inactief en laat een werkend brein structuurloos lijken. Dat is geen theoretisch punt: in een eerdere versie van de meetcode gebeurde precies dat.

Kortste pad van zintuig naar knop

De lengte in bogen van het kortste pad in E_θ van een willekeurige invoerknoop naar een willekeurige uitvoerknoop. Twee is het minimum dat de regels toelaten. Bestaat er geen pad, dan is het brein per definitie stuurloos — een diagnose die je aan de prestatie alleen niet zou zien.

Lussen

Het aantal sterk samenhangende componenten van meer dan één knoop in de actieve deelgraaf, plus het aantal actieve zelfverbindingen van geheugen-neuronen. Dit telt hoeveel echte terugkoppeling er in het netwerk zit.

Werkende reflexbogen

Het aantal reflex-neuronen dat zowel vanaf de invoer bereikbaar is als een actieve uitgang heeft. Dit is de maat voor "heeft dit brein een korte weg van zintuig naar spier gevormd".

Deelnemende neuronen

Een neuron doet mee als het bereikbaar is vanaf de invoer én de uitvoer kan bereiken. De rest is losgeraakt: het draait wel mee in de berekening, maar kan het gedrag onmogelijk beïnvloeden. Het verschil tussen "aantal neuronen" en "aantal meedoende neuronen" is de eerlijkste maat voor hoe groot een netwerk werkelijk is.

Sensorinvloed

Voor elke invoerknoop sommeren wij de gewichtsmassa die haar via paden van hoogstens vijf bogen bereikt aan de uitvoer:

$$\nu_i = \sum_{h=1}^5 \sum_{o \in \mathcal{O}} (\widetilde{W}^h)_{io}, \quad \widetilde{W}_{ab} = \min\left(1, \frac{|w_{ab}|}{w_{\max}}\right) \quad (34)$$

Dit is een afgeknotte padsom, verwant aan Katz-centraliteit. Zij maakt zichtbaar welke zintuigen het netwerk feitelijk gebruikt en welke het volledig negeert — informatie die uit de prestatie alleen nooit af te lezen is.

7 Implementatie

Het model, de meetscripts en dit document zijn tot stand gekomen met behulp van een generatief AI-hulpmiddel (Claude, Opus 5, Anthropic). Wat dat hulpmiddel precies heeft gedaan, en waarom het niet als auteur is opgevoerd, staat in de verklaring achter in dit document. Het wordt hier genoemd omdat het bij de werkwijze hoort en niet bij de kleine lettertjes.

7.1 Indexering en datastructuren

De knopen liggen in een vaste volgorde: eerst de zestien invoerknopen, dan de vier uitvoerknopen, dan de wolk. Die volgorde is met opzet gekozen. Zij zorgt dat een neuron dat er tijdens het trainen bij komt achteraan wordt toegevoegd, zodat geen enkel bestaand nummer verschuift en geen enkele opgeslagen verbinding hoeft te worden omgenummerd. De verbindingen zelf staan in vier parallelle getypeerde arrays — bron, doel, gewicht, spoor — wat de propagatie een enkele lus over aaneengesloten geheugen maakt.

7.2 Complexiteit

Per spelstap en per poging:

$$C_{\text{tik}} = \mathcal{O}(P(|E| + |V|)), \quad C_{\text{poging}} = \mathcal{O}(TP(|E| + |V|)) \quad (35)$$

De dominante term is het aantal verbindingen, niet het aantal neuronen: een wolk van zestig neuronen met tweeduizend verbindingen rekent trager dan een wolk van tweehonderd neuronen met vijfhonderd. Het herstructureren is lineair in het aantal verbindingen op de soorttoewijzing na, die per kandidaat-neuron één keer over de verbindingen loopt. De krachtenmodel-lay-out voor de weergave is kwadratisch in het aantal knopen en wordt daarom alleen bij structuurveranderingen opnieuw uitgevoerd.

7.3 Numerieke waarborgen

Alle activaties liggen door tanh en σ binnen begrensde intervallen; alle gewichten en biassen zijn hard begrensd op $\pm w_{\max}$; de advantage is begrensd; de sporen zijn genormaliseerd. Er is geen plek in het model waar een grootheid ongelimiteerd kan groeien, wat verklaart waarom er ook bij duizenden pogingen geen numerieke ontsparing optreedt.

7.4 Reproduceerbaarheid

Een run wordt volledig vastgelegd door twee zaadwaarden. Het **breinzaad** bepaalt de startwolk: welke neuronen welke soort krijgen, waar de eerste verbindingen liggen en met welke gewichten zij beginnen. Het **wereldzaad** bepaalt de reeks werelden en, via een tweede deterministische generator, alle ruis tijdens het

leren — de perturbaties in de wolk, de loting van de vier knoppen en elke keuze in de herstructurering. De toets op onbekende werelden heeft een eigen, vaste generator, zodat toetsen de training niet verstoort en zelf herhaalbaar is. Nergens in het leertraject wordt de generator van de omgeving aangeroepen.

Dat is geen ontwerpvens maar een gecontroleerde eigenschap. Dezelfde twee zaden worden twee keer achter elkaar gedraaid, waarna de volledige leercurve, alle tussentijdse toetsen, alle structuurmaten en het volledige eindnetwerk — knoop voor knoop en verbinding voor verbinding — bit voor bit worden vergeleken.

Elke afgeronde training wordt weggeschreven met alle instellingen, beide zaden, de geschiedenis per poging, de toetsen op onbekende werelden, het logboek van herstructureringen en de volledige eindstructuur van het netwerk — alle knopen met hun soort, hun positie en hun bias, en alle verbindingen met hun gewicht. Bijlage B geeft de volledige veldenlijst. Zo'n bestand is niet alleen leesbaar maar ook laadbaar: het netwerk kan er ongewijzigd uit teruggebouwd worden om opnieuw getoetst te worden, om verder te trainen, of om als startpunt voor een variant te dienen. Dat maakt de meetopstelling ook geschikt voor reeksen: een lijst condities maal zaden wordt achter elkaar afgedraaid en levert per run een bestand en één regel in een gezamenlijke tabel.

8 Parameters

Onderstaande tabel vat de vrije parameters samen, met de waarden die als standaard gelden. De laatste kolom geeft aan waar de parameter in het model optreedt.

Symbool	Betekenis	Standaard	Zie
n	aantal neuronen bij de start	60	sectie 2.1
d	startdichtheid: uitgaande verbindingen per neuron	35	sectie 4.2
P	propagatiestappen per spelstap	1	vgl. 4–6
α	geheugenlek	0,15	vgl. 6
η_0	leersnelheid	0,008	vgl. 17
λ	sporenvervaging	0,60	vgl. 14
β	aanpassingssnelheid van de basislijn	0,02	vgl. 15
$w_{\max}$	maximaal gewicht	2,0	vgl. 18, 20
$\Delta_{\max}$	maximale verandering per stap	0,100	vgl. 19
ρ	vervaging per poging	0,0001	vgl. 21
ε_0	exploratie bij aanvang	0,30	vgl. 22
ε_{∞}	ondergrens exploratie	$\max(0,3 \varepsilon_0 ; 0,03)$	vgl. 22
K	pogingen tussen twee herstructureringen	10	sectie 4
θ_p	snoeidrempel	0,030	vgl. 24
—	nieuwe verbindingen per ronde	6	sectie 4.2
$n_{\max}$	plafond op het aantal neuronen	140	sectie 4.3
—	nieuwe neuronen per ronde	2	sectie 4.3
$C_{\max}$	maximale omschakelkosten	0,25	vgl. 27
—	obstakels in de wereld	7	sectie 5
T	stappen per poging	320	sectie 5
E	aantal pogingen	500	algoritme 1
—	werelden per toets	20	sectie 5.4

9 Beperkingen

- **De gradiëntschatter is ruis.** Node-perturbatie levert een zuivere maar variabele schatting; de variantie groeit met het aantal knopen dat tegelijk verstoord wordt. Dat begrenst hoe groot een wolk zinvol kan worden zonder de leersnelheid te verlagen.
- **De twee delen van de update staan niet op dezelfde schaal.** Sectie 3.11 meet dat het node-perturbatiedeel twee ordes te klein is ten opzichte van het score-functiedeel, doordat de normalisatie $1/\text{Var}(\xi)$ ontbreekt. Met één leersnelheid krijgt de wolk daardoor nauwelijks een update. Dit is de scherpste bekende tekortkoming van de leerregel; het herstellen ervan vraagt om de leersnelheid en de stapbegrenzing samen opnieuw af te stellen, en dat is nog niet gedaan.
- **De uitkomst hangt af van het punt in de gewichtsruimte.** De cosinus van het wolkdeel met de echte gradiënt loopt in dezelfde opstelling van 0,27 bij een vers brein tot 0,83 na tweehonderd pogingen leren. Een meting op één punt zegt dus weinig; de controle is uitgevoerd op drie punten en zelfs dat is weinig.
- **De controle is klein.** De numerieke gradiënt is alleen na te rekenen op een graaf met vier verborgen knopen en een bevroren topologie, in een omgeving waarin die miniatuuragent het doel zelden haalt. Dat J daar vooral door de dichtheidsterm wordt bepaald is geen bezwaar voor het toetsen van de leerregel — dat is ook het signaal dat het brein tijdens de training grotendeels ziet — maar het blijft een ander regime dan het volledige model.
- **Geen kritiekfunctie.** De advantage is een lopend gemiddelde over stappen, geen toestandsafhankelijke waardeschatting. In toestanden die systematisch beter of slechter zijn dan gemiddeld, is het signaal daardoor vertekend.
- **Soorttoewijzing is lokaal.** Het criterium in vergelijking 26 kijkt alleen naar de bogen van het neuron zelf, niet naar wat de verandering verderop in het netwerk aanricht. Ook de rangorde van specificiteit is een ontwerpkeuze en geen afgeleid resultaat; een criterium dat de doorstroming over paden meeweegt zou principiëler zijn.
- **Groei kent geen krimp.** Neuronen kunnen bijkomen en hun soort verliezen, maar worden nooit verwijderd. Een netwerk dat te ver doorgroeit kan alleen nog verdunnen, niet inkrimpen.
- **De structuurmaten hangen aan een drempel.** Wat "actief" heet, is een keuze: een andere drempel geeft andere padlengtes, andere lussen en andere reflexbogen. De maten zijn geschikt om breinen met elkaar te vergelijken, niet als absolute grootheid.
- **Eén omgeving.** Alle ontwerpbeslissingen zijn gemaakt met één taak voor ogen. Welke ervan algemeen zijn en welke aan die taak vastzitten, is een open vraag.

10 Evaluatie

De meetopzet is met opzet niet ingericht om te laten zien dat het model werkt, maar om uit elkaar te trekken waar een eventuele prestatie vandaan komt. Deze versie van het document bevat de referentiemeting en de reproduceerbaarheidscontrole, en sectie 3.11 de numerieke controle van de leerregel zelf; de ablaties en de basislijnen volgen in een latere versie.

10.1 Twee meetassen

Prestatie wordt gescheiden gemeten op de werelden waarin getraind is en op werelden die het netwerk nooit gezien heeft. Het verschil tussen die twee is de eigenlijke grootheid: het onderscheidt het uit het hoofd leren van een route van het leren navigeren. Daarnaast wordt na elke run de gevormde structuur met de maten uit sectie 6 vastgelegd, zodat de vraag beantwoord kan worden welke structurele eigenschappen met goed presteren samenhangen. Als referentiepunt dient een zuiver reactieve agent die recht op het doel af loopt en langs obstakels glijdt; die geeft aan wat zonder geheugen of planning haalbaar is.

Over onzekerheid. De spreiding tussen zaden is bij deze leerregel aanzienlijk, en een gemiddelde zonder interval is daarom misleidend. Alle getallen hieronder staan als gemiddelde met een 95%-interval over de zaden. Overlappen twee intervallen elkaar ruim, dan is een verschil niet aangetoond, hoe suggestief het gemiddelde ook oogt.

10.2 Referentiemeting

De standaardconfiguratie — 60 neuronen bij aanvang, startdichtheid 35, 7 obstakels, 320 stappen per poging, 500 pogingen — is over 12 onafhankelijke breinzaden gedraaid, alle met hetzelfde wereldzaad, zodat de zaden alleen in de startwolk verschillen. Dit is de conditie waartegen elke latere ablatie wordt afgezet.

grootheid	gemiddelde $\pm$ 95%	spreiding (sd)	bereik
succes over alle pogingen	63.5% $\pm$ 1.9	3.4%	60–70%
succes laatste 20 pogingen	89.2% $\pm$ 5.6	10.0%	70–100%
toets op onbekende werelden	67.9% $\pm$ 4.3	7.5%	55–80%
stappen bij een geslaagde poging	160 $\pm$ 5	8	145–170
rekentijd per run (s)	12.9 $\pm$ 0.3	0.5	12–14

Het verschil tussen de laatste twintig trainingspogingen (**89.2% $\pm$ 5.6**) en de toets op werelden die het netwerk nooit gezien heeft (**67.9% $\pm$ 4.3**) is 21.2 procentpunt. Dat gat is de kern van de vraag waaruit dit werk voortkomt. Het is klein genoeg om te concluderen dat er navigatiegedrag geleerd is en niet louter een route onthouden, en groot genoeg om te laten zien dat het onthouden meespeelt. De sd van 10.0% op de trainingsscore is bovendien het getal dat bepaalt hoeveel zaden een latere vergelijking nodig heeft: met deze spreiding is een verschil van tien procentpunt pas boven de ruis bij ruwweg zestien runs per conditie.

De structuur die eruit komt

Dezelfde runs, nu afgelezen met de maten uit sectie 6. Deze tabel is niet illustratief maar de nulmeting waartegen sectie 6 in een volgende versie op prestatie gecorreleerd wordt.

structuurmaat	gemiddelde $\pm$ 95%	bereik
neuronen aan het eind	111 $\pm$ 2	106–116
meedoende neuronen	104 $\pm$ 4	88–115
verbindingen	3001 $\pm$ 41	2905–3157
actieve verbindingen	1446 $\pm$ 107	1132–1711
kortste pad zintuig $\rightarrow$ knop	2.00 $\pm$ 0.00	2–2
terugkoppelingen	11.0 $\pm$ 0.0	11–11
werkende reflexbogen	3.8 $\pm$ 1.5	0–8

Twee dingen vallen op. Het kortste pad is bij vrijwel elke run 2: het netwerk bouwt een directe verbinding van zintuig naar knop en houdt die, ondanks dat er niets in de leerregel is dat korte paden beloont. En het aantal reflexbogen loopt van 0 tot 8 — een spreiding die groter is dan het gemiddelde. Reflexbogen ontstaan dus wel, maar niet betrouwbaar; of ze iets bijdragen is precies wat de ablatie zonder reflex-neuronen moet uitwijzen.

10.3 Wat hierna gemeten wordt

De referentiemeting hierboven zegt op zichzelf nog niets over de vraag of de graafstructuur iets bijdraagt. Daarvoor zijn condities nodig die telkens één onderdeel wegnemen, en basislijnen die telkens één aanname wegnemen. De volgende versie van dit document rapporteert:

- **Ablaties.** Tien condities die elk één mechanisme uitzetten — geheugen-neuronen, reflex-neuronen, invoer-neuronen, snoeien, aangroei, neuronale groei, soortverandering, de soortregels zelf, en als uiterste de volledig bevroren structuur waarin alleen de gewichten nog leren.
- **Basislijn met dezelfde leerregel.** Een gelaagd netwerk met hetzelfde parameterbudget, getraind met exact de leerregel uit sectie 3. Dit isoleert wat de structuur bijdraagt; zonder deze conditie is niet uit te sluiten dat de leerregel alleen het werk doet.
- **Basislijnen met backpropagation.** Twee kleine MLP's en een GRU, getraind met een gewone policy-gradient. Dit isoleert wat er wordt opgegeven door geen backpropagation te gebruiken. Dat is een andere vraag dan de vorige, en beide zijn nodig.
- **Rekenkosten, apart geteld.** Sample-efficiëntie (omgevingsstappen tot een drempel), rekenefficiëntie (kanten-bezoeken tot diezelfde drempel), wandkloktijd en inferentiekosten worden los gerapporteerd. Zij vallen zelden samen, en één cijfer voor "efficiëntie" verbergt meer dan het laat zien.
- **De schaal van het wolkdeel van de update.** Sectie 3.11 laat zien dat het node-perturbatiedeel twee ordes te klein is ten opzichte van het score-functiedeel. De ontbrekende factor toevoegen is één regel code, maar vraagt om de leersnelheid en de stapbegrenzing samen opnieuw af te stellen; dat wordt als volwaardige conditie gemeten, niet als aanname doorgevoerd.
- **Een tweede taak.** In de huidige taak is het doel altijd zichtbaar; er valt niets te onthouden, en elke uitspraak over de geheugen-neuronen is daarmee betekenisloos. Een tweede taak waarin het doel na verloop van tijd verdwijnt terwijl er tegelijk obstakels opduiken die binnen één tik ontwaken moeten worden, is de eerste opzet waarin de twee soorten paden — kort en reflexmatig, lang en met geheugen — ook werkelijk allebei nodig zijn.

Bij dat laatste hoort een afbakening die dit document eerder te ruim liet. ANG is geen goedkoper alternatief voor backpropagation, en op een taak als deze zal een klein gelaagd netwerk vermoedelijk op rekenkosten winnen. De verdedigbare vraag is smaller: of een lokaal lerende, zichzelf herstructurerende recurrente graaf door structurele spaarzaamheid en verschillende informatielatenties een gunstiger compromis tussen rekenwerk en gedrag vindt dan een vaste architectuur. Bij propagatiediepte één kost elke boog letterlijk één tijdstap, zodat padlengte in dit model samenvalt met reactietijd — en dat is de eigenschap die een vaste architectuur niet vanzelf heeft.

11 Conclusie

ANG laat zien dat de laagstructuur van een neurale netwerk opgegeven kan worden zonder dat er iets voor in de plaats hoeft te komen wat even star is. Wat overblijft is een getypeerde graaf met vijf invarianten, een leerregel die volledig uit lokale grootheden is opgebouwd, en een herstructureringsprocedure die de topologie en zelfs de soort van een neuron laat volgen uit wat er feitelijk gegroeid is. Drie begrenzings — niet abrupt, niet oneindig, en vervagend — vervangen de rol die een expliciete optimalisator in een gelaagd netwerk speelt.

De prijs is bekend: een gradiëntschatter met variantie, geen convergentiegarantie, en een gevoeligheid voor parameters die in een gelaagd netwerk niet bestaat. De winst is dat de uitkomst afleesbaar is. Aan het einde van een run staat er geen gewichtsmatrix maar een structuur, met een kortste pad, met lussen, met reflexbogen, met zintuigen die gebruikt worden en zintuigen die genegeerd worden. Voor de vraag waar dit werk uit voortkomt — begrijpen waarom een te klein of slecht getraind netwerk faalt — is dat precies de vorm waarin je een antwoord wilt hebben.

Verklaring over het gebruik van AI-hulpmiddelen

Bij dit onderzoek is Claude (Opus 5, Anthropic) gebruikt, en niet marginaal. Het hulpmiddel staat **niet als auteur vermeld**. Dat is geen bescheidenheid maar de regel: een AI-hulpmiddel kan geen verantwoording dragen voor het werk, geen belangenverstrengeling melden en geen rechten overdragen, en kan daarom

volgens COPE, de ICMJE en het beleid van arXiv geen auteur zijn. De auteur is volledig verantwoordelijk voor de inhoud van dit document, ook voor de delen die met behulp van het hulpmiddel tot stand zijn gekomen.

Waar het aan heeft bijgedragen

- **Ontwerp en implementatie.** Het model, de leerregel en de structurele plasticiteit zijn in samenspraak ontworpen: de auteur stelde het onderzoeksdoel, de eis van een laagloze graaf, de vier neuronsoorten en hun bedradingsregels vast; het hulpmiddel heeft die eisen uitgewerkt tot de formuleringen in de secties 2 tot en met 6 en tot de volledige implementatie in JavaScript.
- **Metten en analyseren.** De meetscripts, de experimentloper, de statistiek en de figuren zijn met het hulpmiddel geschreven en door het hulpmiddel uitgevoerd. Alle ruwe meetgegevens zijn weggeschreven en meegeleverd (zie hieronder), zodat elke uitspraak in dit document tot de onderliggende getallen te herleiden is.
- **Schrijven.** De tekst van dit document is door het hulpmiddel opgesteld en door de auteur inhoudelijk beoordeeld en geaccordeerd.

Wat dat betekent voor de lezer

Twee dingen. Ten eerste: de tabellen in de secties 3.5, 3.11 en 10 worden door de generator rechtstreeks uit de meetbestanden opgebouwd en niet met de hand overgetypt, juist omdat een getal dat een taalmodel uit zijn hoofd opschrijft geen meting is. Ontbreekt een meetbestand, dan zegt de betreffende sectie dat de meting nog moet gebeuren in plaats van een getal te noemen. Ten tweede: de referenties zijn nog niet stuk voor stuk tegen de originelen geverifieerd. Tot dat gebeurd is, moet de lezer de bibliografie met gepaste argwaan lezen — juist bij een document dat op deze manier tot stand is gekomen.

Belangenverklaring

De auteur verklaart geen concurrerende financiële of persoonlijke belangen te hebben die de in dit document gerapporteerde bevindingen hebben kunnen beïnvloeden. Er is geen externe financiering ontvangen. Anthropic, de leverancier van het gebruikte AI-hulpmiddel, heeft geen rol gehad bij de opzet van het onderzoek, de uitvoering, de analyse of het besluit tot publicatie.

Beschikbaarheid van code en gegevens

Alles wat nodig is om de cijfers in dit document na te rekenen is openbaar: github.com/spikalo/AI-testing-tool.

wat	waar
het model en de leeromgeving	brein-test.html — één bestand, geen bibliotheken, draait in de browser
de generator van dit document	paper/paper.js, met paper/mkeq.py en de figuurscripts
de meetreeksen	experimenten/runs.csv — één regel per run, met beide zaden en alle instellingen
de gradiëntcontrole	experimenten/gradcheck.csv en gradcheck*.json
de controles die de eigenschappen bewijzen	tests/ — Playwright-scripts, elk met een eigen uitkomst
de reproduceerbaarheidscontrole	experimenten/reproduceerbaarheid.json

Een run ligt volledig vast door twee getallen: het breinzaad en het wereldzaad, die beide in elk resultaatbestand staan (bijlage B). Twee runs met dezelfde zaden geven bit voor bit dezelfde leercurve, dezelfde toetsen, dezelfde structuurmaten en hetzelfde eindnetwerk — gecontroleerd, niet aangenomen. De volledige resultaatbestanden per run zijn niet meegeleverd omdat zij groot zijn en uit die twee zaden exact te reproduceren.

Referenties

- [1] Bogdan, P. A., Rowley, A. G. D., Rhodes, O., & Furber, S. B. (2018). Structural plasticity on the SpiNNaker many-core neuromorphic system. *Frontiers in Neuroscience*, 12.
- [2] Chklovskii, D. B., Mel, B. W., & Svoboda, K. (2004). Cortical rewiring and information storage. *Nature*, 431(7010), 782–788.
- [3] Fiete, I. R., & Seung, H. S. (2006). Gradient learning in spiking neural networks by dynamic perturbation of conductances. *Physical Review Letters*, 97(4), 048104.
- [4] Frémaux, N., & Gerstner, W. (2016). Neuromodulated spike-timing-dependent plasticity and theory of three-factor learning rules. *Frontiers in Neural Circuits*, 9, 85.
- [5] Holtmaat, A., & Svoboda, K. (2009). Experience-dependent structural synaptic plasticity in the mammalian brain. *Nature Reviews Neuroscience*, 10(9), 647–658.
- [6] Izhikevich, E. M. (2007). Solving the distal reward problem through linkage of STDP and dopamine signaling. *Cerebral Cortex*, 17(10), 2443–2452.
- [7] Jaeger, H. (2001). The "echo state" approach to analysing and training recurrent neural networks. GMD Report 148.
- [8] Maass, W., Natschläger, T., & Markram, H. (2002). Real-time computing without stable states: a new framework for neural computation based on perturbations. *Neural Computation*, 14(11), 2531–2560.
- [9] Ng, A. Y., Harada, D., & Russell, S. (1999). Policy invariance under reward transformations: theory and application to reward shaping. *Proceedings of ICML*, 278–287.
- [10] Seung, H. S. (2003). Learning in spiking neural networks by reinforcement of stochastic synaptic transmission. *Neuron*, 40(6), 1063–1073.
- [11] Stanley, K. O., & Miikkulainen, R. (2002). Evolving neural networks through augmenting topologies. *Evolutionary Computation*, 10(2), 99–127.
- [12] Sutton, R. S., & Barto, A. G. (2018). *Reinforcement Learning: An Introduction* (2e druk). MIT Press.
- [13] Williams, R. J. (1992). Simple statistical gradient-following algorithms for connectionist reinforcement learning. *Machine Learning*, 8, 229–256.
- [14] Xie, S., Kirillov, A., Girshick, R., & He, K. (2019). Exploring randomly wired neural networks for image recognition. *Proceedings of ICCV*, 1284–1293.
- [15] You, J., Leskovec, J., He, K., & Xie, S. (2020). Graph structure of neural networks. *Proceedings of ICML*, 10881–10891.

Bijlage A — Notatie

Symbool	Betekenis
V, E	knopen en gerichte verbindingen van de graaf
I, O, H	invoerknopen, uitvoerknopen, de wolk ertussen
$\kappa(v)$	de soort van knoop v
$L(\cdot, \cdot)$	toelaatbaarheidsrelatie tussen soorten
w_{ij}, b_j	gewicht van verbinding $i \rightarrow j$; bias van knoop j
u_j, x_j	netto-invoer en activatie van knoop j
$\tilde{x}_i$	presynaptische activatie: de toestand aan het begin van de laatste propagatiestap
$\tilde{x}_j, \delta_j$	schone activatie en perturbatie ($= x - \bar{x}$)
p_k, a_k	kans op en uitkomst van knopdruk k
e_{ij}	eligibility trace van verbinding $i \rightarrow j$
$r_t, \tilde{r}_t, \hat{A}_t$	beloning, lopende basislijn, advantage
$\varepsilon, \tau, \sigma_n$	exploratie, temperatuur van de knoppen, ruis in de wolk
η, λ, ρ	leersnelheid, sporenvervaging, vervaging per poging
φ	dempingsfactor tegen het gewichtsplafond
θ_p, n_{ij}	snoeidrempel en het aantal ronden dat $i \rightarrow j$ al zwak is
$C(i, t), S(t)$	kosten van soort t voor neuron i ; de specificiteitsbonus

Symbool	Betekenis
$T(i), \mu$	de gekozen soort; de marge waarmee zij moet winnen
v_i	invloed van invoerknoop i op de uitvoer

Bijlage B — Het resultaatbestand

Elke afgeronde run wordt als één JSON-bestand weggeschreven. De indeling is zo gekozen dat het bestand op zichzelf voldoende is: er is geen aanvullend logboek nodig om de run te herhalen, het netwerk te hertekenen of het brein terug te laden.

veld	inhoud
versie	formaatversie van het bestand
tijdstip, spel, brein	wanneer, welke taak, welke naam
breinZaad	zaad van de startwolk — samen met het wereldzaad legt dit de run vast
config.startNeuronen, .samenstelling, .samenstellingStart	aantal en soortverdeling bij begin en einde
config.startdichtheid, .invoerAlleenViaSens	bedradingsdichtheid en de strengere invoerregel
config.types	per soort: aan/uit met minimum en maximum
config.wereld	obstakels, stappen per poging, verversing, wereldzaad, snelheid, stoppen bij botsing
config.leren	$\eta, \lambda, \rho, w_{\max}, \Delta_{\max}$, geheugenlek, propagatiediepte, exploratie begin en eind
config.structureel	ritme, snoeidrempel, aangroei, groei, soortverandering en hun grenzen
config.beloningen	doel, vooruitgang, botsing, per stap, stilstaan
config.training	pogingen, toetsen aan/uit, toetsmodus, aantal toetswerelden
resultaat	succes totaal en over de laatste twintig, toetsscore, stappen, beloning, botsingen, rekentijd
structuur	alle maten uit sectie 6, plus gesnoeid, bijgegroeid, geboren en omgetypt
historie	per poging: geslaagd, beloning, stappen, stilstaan, verbindingen, neuronen, gemiddeld gewicht
toetsen	per toetsmoment de score op de onbekende werelden
herstructureringen	per ronde wat er gesnoeid, aangegroeid, omgetypt en bijgegroeid is
netwerk.neuronen	per knoop: nummer, soort, label, positie, bias
netwerk.verbindingen	per verbinding: bron, doel, gewicht
netwerk.nextId	volgnummer voor labels van later geboren neuronen

Een reeks runs schrijft daarnaast één regel per run naar een gezamenlijke tabel met 53 kolommen: de conditiennaam, beide zaden, de parameters die tussen condities kunnen verschillen, en alle uitkomst- en structuurmaten. Die tabel is de bron waaruit de tabellen in sectie 10 van dit document worden opgebouwd — niet met de hand overgetypt, maar bij elke hergeneratie opnieuw uit de meetgegevens gelezen.